Day 4 — Planner/Executor Architecture

> 🎯 **Goal**: Build a proper planning agent capable of decomposing tasks.

> ⏱️ **Time**: 1–2h theory + 2–3h practice

> 📦 **Deliverable**: Working Planner/Executor system with a Critic loop

---

Table of Contents

1. Why Planning Matters
2. Task Decomposition
3. Planner → Executor Pipeline
4. Self-Correction: The Critic Loop
5. Guardrails
6. Practice: Build the Full System

Why Planning Matters

Day 3's ReAct agent decides **one step at a time**. This works for simple tasks but fails for complex ones:

```
ReAct (step-by-step):
  "Build a web scraper that extracts prices from 3 e-commerce sites"

  Step 1: Hmm, let me start by scraping Amazon...
  Step 2: OK, got Amazon data. Now let me try eBay...
  Step 3: eBay has a different format... let me figure it out...
  Step 4: Done with eBay. Now what was the third site?  ← Lost track!
  Step 5: Oh wait, I should have extracted PRICES, not just HTML...

  Problem: No upfront plan → wasted steps, forgotten requirements

Planner/Executor (plan-first):
  PLANNER generates:
    Step 1: Identify 3 target e-commerce sites
    Step 2: For each site, determine the HTML structure for prices
    Step 3: Write extraction logic for each site
    Step 4: Run extractors and collect prices
    Step 5: Format results as comparison table
    Step 6: Validate all 3 sites returned data
```

```
  EXECUTOR runs each step.
  CRITIC validates after each step.

  Result: Organized, complete, no wasted effort.
```

When to Use Planning vs ReAct

```
┌──────────────────────────┬─────────────┬──────────────────────┐
│ Task Type                │ Use ReAct   │ Use Planner/Exec     │
├──────────────────────────┼─────────────┼──────────────────────┤
│ Simple Q&A               │ ✓           │                      │
│ Single tool lookup       │ ✓           │                      │
│ 2-3 step tasks           │ ✓           │                      │
│ Multi-step research      │             │ ✓                    │
│ Tasks with dependencies  │             │ ✓                    │
│ Code generation + test   │             │ ✓                    │
│ Document analysis        │             │ ✓                    │
│ Complex data pipelines   │             │ ✓                    │
└──────────────────────────┴─────────────┴──────────────────────┘
```

Task Decomposition

Task decomposition is the art of breaking a complex task into smaller, manageable subtasks.

Decomposition Strategies

```
Strategy 1: SEQUENTIAL DECOMPOSITION
  Break into ordered steps where each depends on the previous.

  Task: "Analyze our competitors' pricing"
  Decomposition:
    1. Identify top 5 competitors  (independent)
    2. Collect pricing data         (depends on 1)
    3. Normalize pricing formats    (depends on 2)
    4. Create comparison matrix     (depends on 3)
    5. Write analysis report        (depends on 4)

Strategy 2: PARALLEL DECOMPOSITION
  Break into independent subtasks that can run simultaneously.

  Task: "Analyze weather in 3 cities"
  Decomposition:
    1a. Get weather for Tokyo     (independent)
    1b. Get weather for London    (independent)
    1c. Get weather for New York  (independent)
    2. Compare results            (depends on 1a, 1b, 1c)
```

```
Strategy 3: HIERARCHICAL DECOMPOSITION
  Break into categories, then sub-steps within each.

  Task: "Create a user registration system"
  Decomposition:
    1. Backend
       1.1 Design database schema
       1.2 Create API endpoints
       1.3 Add input validation
    2. Frontend
       2.1 Design registration form
       2.2 Add client-side validation
       2.3 Connect to API
    3. Testing
       3.1 Unit tests for API
       3.2 Integration tests
       3.3 E2E tests
```

The Decomposition Prompt

```
  PLANNER_PROMPT = """You are an expert task planner. Given a complex task,
  break it down into clear, actionable steps.

  ## Rules for Planning
  1. Each step must be concrete and actionable (not vague)
  2. Steps should be in dependency order
  3. Each step should be completable by a single tool or action
  4. Include validation steps to verify correctness
  5. Mark dependencies explicitly

  ## Available Tools
  {tools_description}

  ## Output Format
  Respond with a JSON object:
  {{
    "task_understanding": "Brief restatement of the task",
    "approach": "High-level approach in 1-2 sentences",
    "steps": [
      {{
        "id": 1,
        "description": "What to do",
        "tool": "tool_name or 'reasoning'",
        "inputs": {{}},
        "depends_on": [],
        "expected_output": "What success looks like",
        "validation": "How to verify this step succeeded"
      }}
    ],
    "success_criteria": "How to know the entire task is complete"
  }}

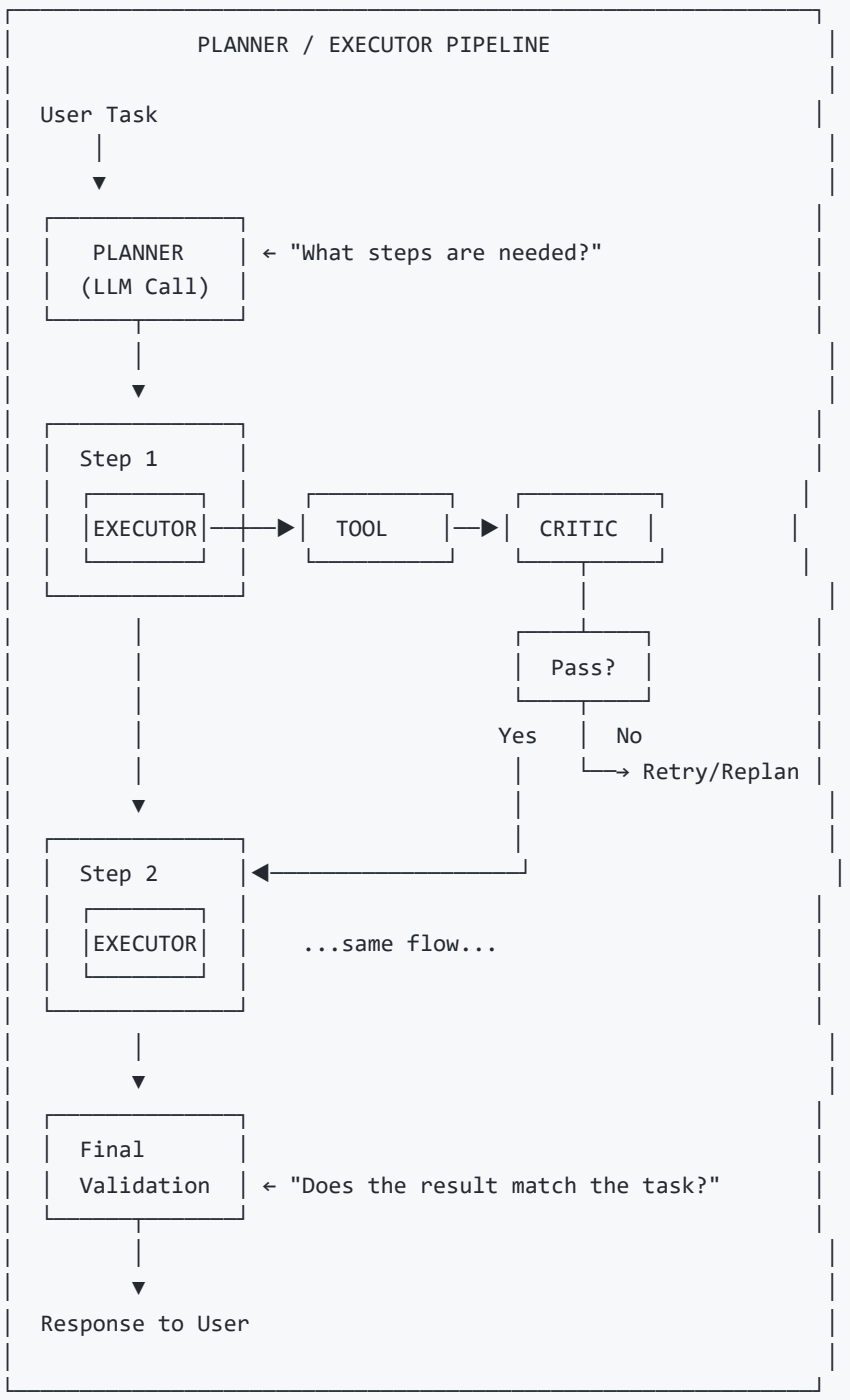
  ## Task
```

```
{task}

Plan:"""
```

Planner → Executor Pipeline

The Planner/Executor architecture separates **thinking** from **doing**:



Component Responsibilities

```
PLANNER:
  Input:  User's task + available tools
  Output: Ordered list of steps with dependencies
  LLM:    Needs strong reasoning (GPT-4, Claude 3)
  Temp:   Low (0.0-0.2) for consistent plans

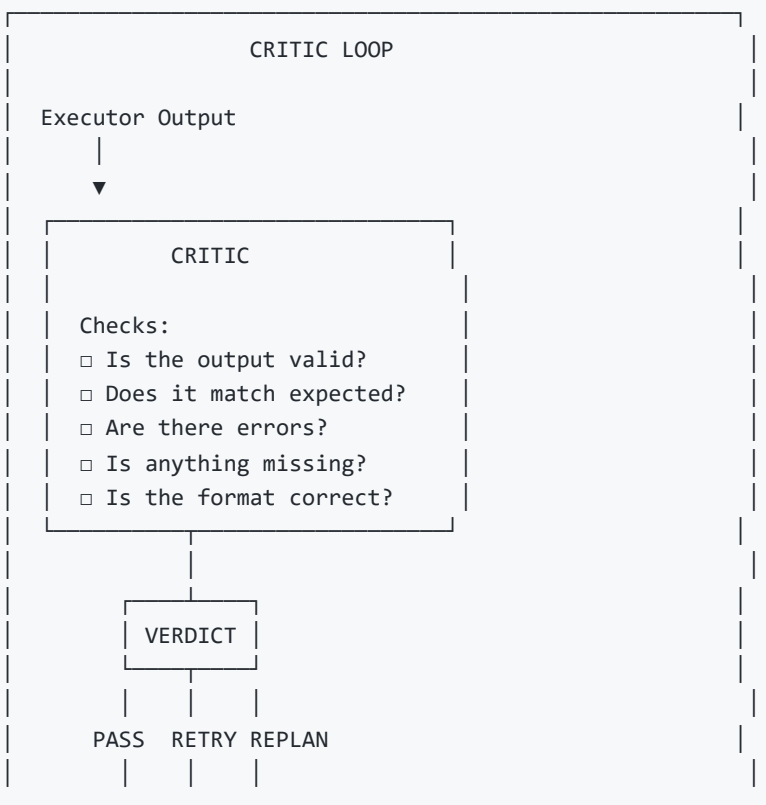
EXECUTOR:
  Input:  Single step from the plan + available tools
  Output: Step result + status (success/failure)
  LLM:    Can use lighter model (GPT-3.5, Llama 8B)
  Temp:   Low (0.0) for reliable execution

CRITIC:
  Input:  Step description + expected output + actual output
  Output: Pass/Fail + reasoning + suggestions
  LLM:    Needs good judgment (GPT-4, Claude 3)
  Temp:   Low (0.0) for consistent evaluation
```

Self-Correction: The Critic Loop

The Critic is what makes the system **reliable**. Without it, errors cascade through the pipeline.

The Critic Loop



```
│        │    │    └──→ Send back to PLANNER              │
│        │    │         "Step 3 failed because X.         │
│        │    │          Please create a new plan          │
│        │    │          starting from step 3."            │
│        │    │                                           │
│        │    └──→ Send back to EXECUTOR                  │
│        │         "Result was wrong because X.           │
│        │          Try again with this guidance: Y"      │
│        │                                                │
│        └──→ Continue to next step                      │
│                                                         │
└─────────────────────────────────────────────────────────┘
```

Critic Prompt

```
CRITIC_PROMPT = """You are a quality assurance critic. Your job is to evaluate
whether a step was executed correctly.

## Step Details
- Description: {step_description}
- Expected Output: {expected_output}
- Validation Criteria: {validation_criteria}

## Actual Output
{actual_output}

## Evaluate

Respond with a JSON object:
{{
  "verdict": "PASS" | "RETRY" | "REPLAN",
  "reasoning": "Why you made this decision",
  "issues": ["List of specific issues found"],
  "suggestions": ["How to fix or improve"],
  "confidence": 0.0-1.0
}}

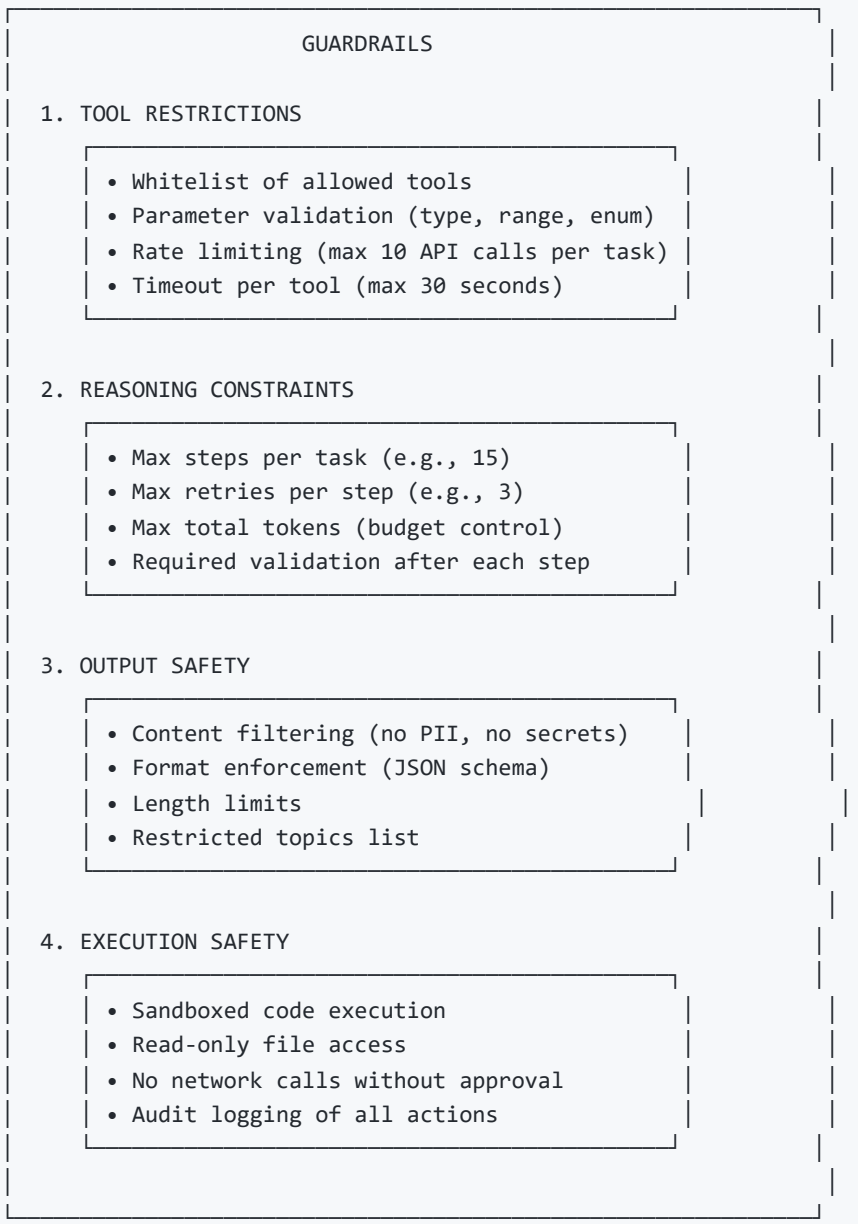
Rules:
- PASS: Output is correct and complete
- RETRY: Output has minor issues that can be fixed by re-executing the step
- REPLAN: Output is fundamentally wrong; the plan needs to change

Be strict but fair. Minor formatting issues = PASS with suggestions.
Missing data or wrong answers = RETRY or REPLAN."""
```

Guardrails

Guardrails prevent the agent from doing harmful or unintended things.

Types of Guardrails



Implementing Guardrails

```python
from dataclasses import dataclass, field
from typing import Optional
import time

@dataclass
class GuardrailConfig:
    """Configuration for agent guardrails."""
    max_steps: int = 15
    max_retries_per_step: int = 3
    max_total_retries: int = 10
    tool_timeout_seconds: float = 30.0
```

```
    max_tool_calls: int = 50
    allowed_tools: Optional[list] = None  # None = all tools allowed
    blocked_tools: list = field(default_factory=list)
    max_output_length: int = 5000
    require_critic: bool = True


class GuardrailEnforcer:
    """Enforces guardrails during agent execution."""

    def __init__(self, config: GuardrailConfig):
        self.config = config
        self.step_count = 0
        self.retry_counts = {}  # step_id → retry count
        self.total_retries = 0
        self.tool_call_count = 0
        self.audit_log = []

    def check_step_limit(self) -> tuple:
        """Check if we've exceeded the step limit."""
        self.step_count += 1
        if self.step_count > self.config.max_steps:
            return False, f"Maximum steps ({self.config.max_steps}) exceeded"
        return True, ""

    def check_retry_limit(self, step_id: str) -> tuple:
        """Check if we've exceeded retries for a step."""
        self.retry_counts[step_id] = self.retry_counts.get(step_id, 0) + 1
        self.total_retries += 1

        if self.retry_counts[step_id] > self.config.max_retries_per_step:
            return False, f"Max retries ({self.config.max_retries_per_step}) for ste

        if self.total_retries > self.config.max_total_retries:
            return False, f"Max total retries ({self.config.max_total_retries}) exce

        return True, ""

    def check_tool_allowed(self, tool_name: str) -> tuple:
        """Check if a tool is allowed."""
        if tool_name in self.config.blocked_tools:
            return False, f"Tool '{tool_name}' is blocked"

        if self.config.allowed_tools and tool_name not in self.config.allowed_tools:
            return False, f"Tool '{tool_name}' not in allowed list"

        self.tool_call_count += 1
        if self.tool_call_count > self.config.max_tool_calls:
            return False, f"Max tool calls ({self.config.max_tool_calls}) exceeded"

        return True, ""

    def execute_with_timeout(self, func, *args, **kwargs):
        """Execute a function with a timeout."""
        import threading
```

```
            result = [None]
            error = [None]

            def target():
                try:
                    result[0] = func(*args, **kwargs)
                except Exception as e:
                    error[0] = e

            thread = threading.Thread(target=target)
            thread.start()
            thread.join(timeout=self.config.tool_timeout_seconds)

            if thread.is_alive():
                return None, f"Tool timed out after {self.config.tool_timeout_seconds}s"

            if error[0]:
                return None, f"Tool error: {error[0]}"

            return result[0], ""

        def validate_output(self, output: str) -> tuple:
            """Validate output against guardrails."""
            if len(output) > self.config.max_output_length:
                return False, f"Output too long ({len(output)} > {self.config.max_output
            return True, ""

        def log_action(self, action: str, details: dict):
            """Log an action for audit purposes."""
            self.audit_log.append({
                "timestamp": time.time(),
                "action": action,
                "details": details
            })

        def get_summary(self) -> dict:
            """Get a summary of guardrail enforcement."""
            return {
                "total_steps": self.step_count,
                "total_retries": self.total_retries,
                "tool_calls": self.tool_call_count,
                "retry_counts": dict(self.retry_counts),
                "audit_log_size": len(self.audit_log)
            }
```

Practice: Build the Full System

Complete Planner/Executor/Critic Implementation

```
"""
Planner/Executor/Critic Agent System

Architecture:
   ┌─────────┐     ┌─────────┐     ┌─────────┐
   │ PLANNER │────▶│ EXECUTOR│────▶│  CRITIC │
   │         │     │         │     │         │
   │ Breaks  │     │ Runs    │     │ Validates│
   │ task into│     │ each    │     │ each    │
   │ steps   │     │ step    │     │ result  │
   └─────────┘     └─────────┘     └─────────┘
        ▲                               │
        │          ┌─────────┐          │
        └──────────│ REPLAN  │◀─────────┘
                   │ if needed│    (on failure)
                   └─────────┘
"""
import json
import re
from dataclasses import dataclass
from typing import Optional


# ═══════════════════════════════════════════════════════════════
# DATA STRUCTURES
# ═══════════════════════════════════════════════════════════════

@dataclass
class PlanStep:
    """A single step in a plan."""
    id: int
    description: str
    tool: str  # tool name or "reasoning"
    inputs: dict
    depends_on: list
    expected_output: str
    validation: str
    status: str = "pending"  # pending, executing, passed, failed, skipped
    result: Optional[str] = None
    retry_count: int = 0


@dataclass
class CriticVerdict:
    """Critic's evaluation of a step."""
    verdict: str  # PASS, RETRY, REPLAN
    reasoning: str
    issues: list
    suggestions: list
    confidence: float


@dataclass
class Plan:
```

```
    """A complete execution plan."""
    task: str
    approach: str
    steps: list  # List[PlanStep]
    success_criteria: str
    version: int = 1


# ============================================================
# TOOLS (reuse from Day 3 + additions)
# ============================================================

import math

def calculator(expression: str) -> str:
    try:
        allowed = {"sqrt": math.sqrt, "pow": pow, "abs": abs,
                   "round": round, "min": min, "max": max}
        result = eval(expression, {"__builtins__": {}}, allowed)
        return str(result)
    except Exception as e:
        return f"Error: {e}"

def search(query: str) -> str:
    knowledge = {
        "python web frameworks": "Popular Python web frameworks: Django (full-featur
        "rest api best practices": "REST API best practices: Use nouns for resources
        "database comparison": "PostgreSQL: Best for complex queries, ACID complianc
        "authentication methods": "Authentication methods: JWT (stateless, scalable)
    }

    query_lower = query.lower()
    for key, value in knowledge.items():
        if any(word in query_lower for word in key.split()):
            return value

    return f"No results for '{query}'. Try: {', '.join(knowledge.keys())}"

def code_generator(spec: str) -> str:
    """Generate code based on a specification (simulated)."""
    if "hello world" in spec.lower():
        return 'def hello():\n    print("Hello, World!")\n\nhello()'
    if "fibonacci" in spec.lower():
        return ('def fibonacci(n):\n'
                '    if n <= 1:\n'
                '        return n\n'
                '    a, b = 0, 1\n'
                '    for _ in range(2, n + 1):\n'
                '        a, b = b, a + b\n'
                '    return b')
    if "api endpoint" in spec.lower() or "fastapi" in spec.lower():
        return ('from fastapi import FastAPI, HTTPException\n'
                'from pydantic import BaseModel\n\n'
                'app = FastAPI()\n\n'
                'class Item(BaseModel):\n'
                '    name: str\n'
```

```
                '    price: float\n\n'
                'items_db = []\n\n'
                '@app.post("/items")\n'
                'async def create_item(item: Item):\n'
                '    items_db.append(item)\n'
                '    return {"id": len(items_db), "item": item}\n\n'
                '@app.get("/items")\n'
                'async def list_items():\n'
                '    return items_db')
    return f"# Generated code for: {spec}\n# TODO: Implement"

def code_reviewer(code: str) -> str:
    """Review code for issues (simulated)."""
    issues = []

    if "eval(" in code:
        issues.append("CRITICAL: Use of eval() — potential code injection")
    if "password" in code.lower() and "=" in code:
        issues.append("HIGH: Possible hardcoded password")
    if "except:" in code or "except Exception:" in code:
        issues.append("MEDIUM: Broad exception handling — catch specific exceptions"
    if "# TODO" in code:
        issues.append("LOW: TODO comments found — incomplete implementation")
    if not issues:
        issues.append("No critical issues found. Code looks reasonable.")

    return json.dumps({"issues": issues, "count": len(issues)})


TOOLS = {
    "calculator": {"fn": calculator, "desc": "Evaluate math expressions"},
    "search": {"fn": search, "desc": "Search for information"},
    "code_generator": {"fn": code_generator, "desc": "Generate code from a specifica
    "code_reviewer": {"fn": code_reviewer, "desc": "Review code for issues"},
}


# ═══════════════════════════════════════════════════════════════
# THE PLANNER
# ═══════════════════════════════════════════════════════════════

class Planner:
    """
    Creates execution plans from task descriptions.

    In production, this would use an LLM. For learning,
    we use rule-based planning so you can run without API keys.
    """

    def create_plan(self, task: str) -> Plan:
        """Create a plan for the given task."""
        task_lower = task.lower()

        # Route to specialized planners based on task type
        if any(w in task_lower for w in ["build", "create", "implement", "write"]):
            return self._plan_build_task(task)
```

```
        elif any(w in task_lower for w in ["analyze", "compare", "evaluate"]):
            return self._plan_analysis_task(task)
        elif any(w in task_lower for w in ["calculate", "compute", "math"]):
            return self._plan_calculation_task(task)
        else:
            return self._plan_research_task(task)

    def replan(self, original_plan: Plan, failed_step: int,
               error: str) -> Plan:
        """Create a new plan when a step fails."""
        # Keep completed steps, redo from failed step
        completed = [s for s in original_plan.steps if s.status == "passed"]

        new_steps = list(completed)
        step_id = len(completed) + 1

        # Add recovery step
        new_steps.append(PlanStep(
            id=step_id,
            description=f"Recover from failure: {error}",
            tool="reasoning",
            inputs={"context": error},
            depends_on=[s.id for s in completed],
            expected_output="Alternative approach identified",
            validation="Recovery plan is actionable"
        ))

        # Add remaining steps with adjusted IDs
        for old_step in original_plan.steps:
            if old_step.status == "pending":
                step_id += 1
                new_steps.append(PlanStep(
                    id=step_id,
                    description=old_step.description,
                    tool=old_step.tool,
                    inputs=old_step.inputs,
                    depends_on=[step_id - 1],
                    expected_output=old_step.expected_output,
                    validation=old_step.validation
                ))

        return Plan(
            task=original_plan.task,
            approach=f"Revised plan after failure at step {failed_step}: {error}",
            steps=new_steps,
            success_criteria=original_plan.success_criteria,
            version=original_plan.version + 1
        )

    def _plan_build_task(self, task: str) -> Plan:
        """Plan for building/creating something."""
        return Plan(
            task=task,
            approach="Research → Design → Implement → Review → Validate",
            steps=[
                PlanStep(
```

```
                    id=1,
                    description="Research best practices and technologies",
                    tool="search",
                    inputs={"query": task},
                    depends_on=[],
                    expected_output="Relevant information about technologies and app
                    validation="Research results are relevant to the task"
                ),
                PlanStep(
                    id=2,
                    description="Generate the code based on requirements and researc
                    tool="code_generator",
                    inputs={"spec": task},
                    depends_on=[1],
                    expected_output="Working code that implements the requirements",
                    validation="Code is syntactically valid and addresses requiremen
                ),
                PlanStep(
                    id=3,
                    description="Review the generated code for issues",
                    tool="code_reviewer",
                    inputs={},  # Will be filled with step 2's output
                    depends_on=[2],
                    expected_output="Code review with no critical issues",
                    validation="No CRITICAL or HIGH severity issues"
                ),
                PlanStep(
                    id=4,
                    description="Compile final result with code and documentation",
                    tool="reasoning",
                    inputs={},
                    depends_on=[3],
                    expected_output="Complete deliverable with code and explanation"
                    validation="All parts of the task are addressed"
                ),
            ],
            success_criteria="Code is generated, reviewed, and has no critical issue
        )

    def _plan_analysis_task(self, task: str) -> Plan:
        """Plan for analysis/comparison tasks."""
        return Plan(
            task=task,
            approach="Gather data → Analyze → Compare → Summarize",
            steps=[
                PlanStep(
                    id=1,
                    description="Gather information about the subject",
                    tool="search",
                    inputs={"query": task},
                    depends_on=[],
                    expected_output="Comprehensive information gathered",
                    validation="Data is relevant and sufficient"
                ),
                PlanStep(
                    id=2,
```

```
                    description="Analyze and organize the findings",
                    tool="reasoning",
                    inputs={},
                    depends_on=[1],
                    expected_output="Organized analysis with key points",
                    validation="Analysis is logical and well-structured"
                ),
                PlanStep(
                    id=3,
                    description="Create final summary with conclusions",
                    tool="reasoning",
                    inputs={},
                    depends_on=[2],
                    expected_output="Clear summary answering the original question",
                    validation="Summary addresses all aspects of the task"
                ),
            ],
            success_criteria="Complete analysis with clear conclusions"
        )

    def _plan_calculation_task(self, task: str) -> Plan:
        """Plan for calculation tasks."""
        # Extract math expressions
        expressions = re.findall(r'[\d\s+\-*/().%]+', task)
        steps = []

        for i, expr in enumerate(expressions, 1):
            expr = expr.strip()
            if expr and any(c.isdigit() for c in expr):
                steps.append(PlanStep(
                    id=i,
                    description=f"Calculate: {expr}",
                    tool="calculator",
                    inputs={"expression": expr},
                    depends_on=[i-1] if i > 1 else [],
                    expected_output=f"Numeric result of {expr}",
                    validation="Result is a valid number"
                ))

        if not steps:
            steps.append(PlanStep(
                id=1,
                description="Parse and calculate the expression",
                tool="calculator",
                inputs={"expression": task},
                depends_on=[],
                expected_output="Numeric result",
                validation="Result is a valid number"
            ))

        return Plan(
            task=task,
            approach="Parse expressions → Calculate → Verify",
            steps=steps,
            success_criteria="All calculations completed with verified results"
        )
```

```
    def _plan_research_task(self, task: str) -> Plan:
        """Default plan for research tasks."""
        return Plan(
            task=task,
            approach="Search → Analyze → Answer",
            steps=[
                PlanStep(
                    id=1,
                    description=f"Search for information about: {task}",
                    tool="search",
                    inputs={"query": task},
                    depends_on=[],
                    expected_output="Relevant information found",
                    validation="Results are relevant to the question"
                ),
                PlanStep(
                    id=2,
                    description="Formulate answer from search results",
                    tool="reasoning",
                    inputs={},
                    depends_on=[1],
                    expected_output="Clear, complete answer",
                    validation="Answer addresses the original question"
                ),
            ],
            success_criteria="Question is fully answered with supporting information
        )


# ============================================================
# THE EXECUTOR
# ============================================================

class Executor:
    """
    Executes individual steps from a plan.
    """

    def __init__(self, tools: dict):
        self.tools = tools
        self.context = {}  # Results from previous steps

    def execute_step(self, step: PlanStep) -> str:
        """Execute a single step and return the result."""
        step.status = "executing"

        if step.tool == "reasoning":
            # Reasoning steps compile context from previous steps
            return self._reason(step)

        if step.tool not in self.tools:
            return f"Error: Tool '{step.tool}' not available"

        # Prepare inputs (may reference previous step results)
        inputs = self._resolve_inputs(step)
```

```
        # Execute the tool
        try:
            tool_fn = self.tools[step.tool]["fn"]

            # Handle different tool signatures
            if step.tool == "code_reviewer" and not inputs.get("code"):
                # Use code from previous code_generator step
                prev_code = self._find_previous_code()
                if prev_code:
                    inputs["code"] = prev_code

            # Call the tool with the first relevant parameter
            if step.tool == "calculator":
                result = tool_fn(inputs.get("expression", str(inputs)))
            elif step.tool == "search":
                result = tool_fn(inputs.get("query", str(inputs)))
            elif step.tool == "code_generator":
                result = tool_fn(inputs.get("spec", str(inputs)))
            elif step.tool == "code_reviewer":
                result = tool_fn(inputs.get("code", str(inputs)))
            else:
                result = tool_fn(**inputs)

            self.context[step.id] = result
            return result

        except Exception as e:
            return f"Error executing {step.tool}: {e}"

    def _reason(self, step: PlanStep) -> str:
        """Perform a reasoning step using accumulated context."""
        context_summary = "\n".join(
            f"Step {sid}: {result[:200]}"
            for sid, result in self.context.items()
        )
        return f"Based on gathered information:\n{context_summary}\n\nConclusion for

    def _resolve_inputs(self, step: PlanStep) -> dict:
        """Resolve step inputs, potentially using previous results."""
        inputs = dict(step.inputs)

        # If inputs reference previous steps, substitute
        for key, value in inputs.items():
            if isinstance(value, str) and value.startswith("$step_"):
                step_id = int(value.replace("$step_", ""))
                if step_id in self.context:
                    inputs[key] = self.context[step_id]

        return inputs

    def _find_previous_code(self) -> Optional[str]:
        """Find code generated by a previous step."""
        for result in reversed(self.context.values()):
            if isinstance(result, str) and ("def " in result or "class " in result):
                return result
```

```
        return None


# ═══════════════════════════════════════════════════════════════
# THE CRITIC
# ═══════════════════════════════════════════════════════════════

class Critic:
    """
    Evaluates step results for correctness and quality.
    """

    def evaluate(self, step: PlanStep, result: str) -> CriticVerdict:
        """Evaluate a step's result."""
        issues = []
        suggestions = []

        # Check for errors
        if result.startswith("Error:"):
            return CriticVerdict(
                verdict="RETRY",
                reasoning=f"Step returned an error: {result}",
                issues=[result],
                suggestions=["Fix the error and retry"],
                confidence=0.9
            )

        # Check for empty results
        if not result or result.strip() == "":
            return CriticVerdict(
                verdict="RETRY",
                reasoning="Step produced empty output",
                issues=["Empty result"],
                suggestions=["Verify inputs and retry"],
                confidence=0.8
            )

        # Check for "no results" from search
        if "no results" in result.lower() or "not found" in result.lower():
            return CriticVerdict(
                verdict="RETRY",
                reasoning="Search returned no results",
                issues=["No information found"],
                suggestions=["Try different search terms",
                             "Broaden the search query"],
                confidence=0.7
            )

        # Check code review results
        if step.tool == "code_reviewer":
            try:
                review = json.loads(result)
                critical = [i for i in review.get("issues", [])
                            if "CRITICAL" in i or "HIGH" in i]
                if critical:
                    return CriticVerdict(
```

```
                        verdict="RETRY",
                        reasoning="Code has critical issues that need fixing",
                        issues=critical,
                        suggestions=["Fix the critical issues before proceeding"],
                        confidence=0.85
                    )
            except json.JSONDecodeError:
                pass

        # Check calculator results
        if step.tool == "calculator":
            try:
                float(result)  # Verify it's a valid number
            except ValueError:
                issues.append(f"Calculator returned non-numeric result: {result}")
                return CriticVerdict(
                    verdict="RETRY",
                    reasoning="Calculator output is not a valid number",
                    issues=issues,
                    suggestions=["Check the expression syntax"],
                    confidence=0.9
                )

        # General quality checks
        if len(result) < 10 and step.tool != "calculator":
            suggestions.append("Result seems brief — consider if more detail is need

        # Default: PASS
        return CriticVerdict(
            verdict="PASS",
            reasoning="Step produced valid, relevant output",
            issues=issues,
            suggestions=suggestions,
            confidence=0.8
        )


# ═══════════════════════════════════════════════════════════════
# THE ORCHESTRATOR
# ═══════════════════════════════════════════════════════════════

class PlannerExecutorSystem:
    """
    Orchestrates the Planner, Executor, and Critic.

    ┌─────────────────────────────────────────────────┐
    │              ORCHESTRATOR                       │
    │                                                 │
    │  Task ──▶ PLANNER ──▶ Plan                      │
    │                        │                        │
    │              ┌─────────┘                        │
    │              │                                  │
    │              ▼                                  │
    │         For each step:                          │
    │              │                                  │
    │              ├──▶ EXECUTOR ──▶ Result           │
```

```
        │              │                   │                │
        │              │             CRITIC ──▶ Verdict     │
        │              │                   │                │
        │              │    PASS ◀─────────┤                │
        │              │    RETRY ◀────────┤                │
        │              │    REPLAN ◀───────┘                │
        │              │                                    │
        │              └──▶ Next step or Replan             │
        │                                                   │
        │  All steps done ──▶ Final Result                  │
        └──────────────────────────────────────────────────┘
    """

    def __init__(self, tools: dict, max_retries: int = 3,
                 max_replans: int = 2):
        self.planner = Planner()
        self.executor = Executor(tools)
        self.critic = Critic()
        self.max_retries = max_retries
        self.max_replans = max_replans

    def run(self, task: str, verbose: bool = True) -> dict:
        """
        Run the full Planner/Executor/Critic pipeline.

        Returns:
            dict with keys: task, plan, results, final_answer, stats
        """
        replan_count = 0

        # Step 1: Create plan
        plan = self.planner.create_plan(task)

        if verbose:
            self._print_plan(plan)

        # Step 2: Execute plan with critic validation
        while replan_count <= self.max_replans:
            success, results = self._execute_plan(plan, verbose)

            if success:
                # Step 3: Compile final answer
                final = self._compile_final_answer(task, plan, results)

                stats = {
                    "total_steps": len(plan.steps),
                    "replans": replan_count,
                    "total_retries": sum(s.retry_count for s in plan.steps),
                    "plan_version": plan.version
                }

                if verbose:
                    self._print_final(final, stats)

                return {
                    "task": task,
```

```
                    "plan": plan,
                    "results": results,
                    "final_answer": final,
                    "stats": stats
                }

            # Replan needed
            replan_count += 1
            if replan_count <= self.max_replans:
                failed_step = next(
                    (s for s in plan.steps if s.status == "failed"), None
                )
                if failed_step:
                    if verbose:
                        print(f"\n⚠ REPLANNING (attempt {replan_count}/{self.max_re
                        print(f"  Failed step: {failed_step.description}")

                    plan = self.planner.replan(
                        plan, failed_step.id,
                        failed_step.result or "Unknown error"
                    )
                    if verbose:
                        self._print_plan(plan)

        return {
            "task": task,
            "plan": plan,
            "results": {},
            "final_answer": "Failed to complete task after maximum replanning attemp
            "stats": {"replans": replan_count}
        }

    def _execute_plan(self, plan: Plan, verbose: bool) -> tuple:
        """Execute all steps in a plan. Returns (success, results)."""
        results = {}

        for step in plan.steps:
            if step.status in ("passed", "skipped"):
                continue

            # Check dependencies
            deps_met = all(
                any(s.id == dep and s.status == "passed"
                    for s in plan.steps)
                for dep in step.depends_on
            )

            if not deps_met and step.depends_on:
                step.status = "skipped"
                if verbose:
                    print(f"\n⏭ Skipping step {step.id}: dependencies not met")
                continue

            # Execute with retry loop
            for attempt in range(self.max_retries + 1):
                if verbose:
```

```
                retry_msg = f" (retry {attempt})" if attempt > 0 else ""
                print(f"\n{'─'*50}")
                print(f"📋 Step {step.id}{retry_msg}: {step.description}")
                print(f"   Tool: {step.tool}")

            # Execute
            result = self.executor.execute_step(step)

            if verbose:
                result_preview = result[:200] + "..." if len(result) > 200 else
                print(f"   Result: {result_preview}")

            # Critic evaluation
            verdict = self.critic.evaluate(step, result)

            if verbose:
                emoji = {"PASS": "✅", "RETRY": "🔄", "REPLAN": "❌"}
                print(f"   Critic: {emoji.get(verdict.verdict, '?')} {verdict.ve
                print(f"   Reason: {verdict.reasoning}")
                if verdict.issues:
                    print(f"   Issues: {verdict.issues}")

            if verdict.verdict == "PASS":
                step.status = "passed"
                step.result = result
                results[step.id] = result
                break

            elif verdict.verdict == "RETRY":
                step.retry_count += 1
                if attempt == self.max_retries:
                    step.status = "failed"
                    step.result = result
                    if verbose:
                        print(f"   ❌ Max retries reached for step {step.id}")
                    return False, results

            elif verdict.verdict == "REPLAN":
                step.status = "failed"
                step.result = result
                return False, results

    return True, results

def _compile_final_answer(self, task: str, plan: Plan,
                          results: dict) -> str:
    """Compile step results into a final answer."""
    parts = [f"Task: {task}\n"]

    for step in plan.steps:
        if step.status == "passed" and step.id in results:
            parts.append(f"• {step.description}:")
            result_text = results[step.id]
            if len(result_text) > 300:
                result_text = result_text[:300] + "..."
            parts.append(f"  {result_text}\n")
```

```
        return "\n".join(parts)

    def _print_plan(self, plan: Plan):
        """Pretty-print a plan."""
        print(f"\n{'='*60}")
        print(f"📝 PLAN (v{plan.version})")
        print(f"Task: {plan.task}")
        print(f"Approach: {plan.approach}")
        print(f"{'─'*60}")
        for step in plan.steps:
            deps = f" (after: {step.depends_on})" if step.depends_on else ""
            status = {"pending": "⬜", "passed": "✅", "failed": "❌",
                      "skipped": "⏭️", "executing": "🔄"}.get(step.status, "?")
            print(f"  {status} Step {step.id}: {step.description}")
            print(f"       Tool: {step.tool}{deps}")
        print(f"Success criteria: {plan.success_criteria}")
        print(f"{'='*60}")

    def _print_final(self, final: str, stats: dict):
        """Pretty-print the final result."""
        print(f"\n{'='*60}")
        print(f"🎯 FINAL RESULT")
        print(f"{'─'*60}")
        print(final)
        print(f"{'─'*60}")
        print(f"Stats: {json.dumps(stats, indent=2)}")
        print(f"{'='*60}")


# ═══════════════════════════════════════════════════════════════
# DEMO
# ═══════════════════════════════════════════════════════════════

def demo():
    """Run demo tasks through the Planner/Executor/Critic system."""
    system = PlannerExecutorSystem(tools=TOOLS)

    print("\n" + "█"*60)
    print(" DEMO 1: Build Task")
    print("█"*60)
    system.run("Build a FastAPI endpoint for a simple CRUD API")

    print("\n\n" + "█"*60)
    print(" DEMO 2: Research Task")
    print("█"*60)
    system.run("What are the best Python web frameworks and why?")

    print("\n\n" + "█"*60)
    print(" DEMO 3: Calculation Task")
    print("█"*60)
    system.run("Calculate 25 * 47 + 330 / 11")

    print("\n\n" + "█"*60)
    print(" DEMO 4: Analysis Task")
    print("█"*60)
```

```
    system.run("Analyze and compare database options for a new project")


if __name__ == "__main__":
    demo()
```

Exercise: Extend the System

```
1. ADD LLM-BASED PLANNER
   Replace the rule-based Planner with an LLM call using
   the PLANNER_PROMPT from the theory section.
   Parse the JSON plan into PlanStep objects.

2. ADD PARALLEL EXECUTION
   Modify the executor to detect steps with no dependencies
   on each other and execute them in parallel (threading).

3. ADD PLAN VISUALIZATION
   Create a function that outputs the plan as an ASCII
   dependency graph:
     Step 1 ──▶ Step 2 ──▶ Step 4
                Step 3 ──▶ Step 4

4. ADD PERSISTENT EXECUTION LOG
   Save each execution to a JSON file with:
   - Task, plan, results, timing
   - Load previous executions to improve future planning
```

✓ Day 4 Deliverable Checklist

- ☐ Built a Planner that decomposes tasks into steps
- ☐ Built an Executor that runs each step using tools
- ☐ Built a Critic that validates step results
- ☐ System handles PASS / RETRY / REPLAN verdicts
- ☐ Guardrails prevent infinite loops and runaway execution
- ☐ Tested with at least 3 different task types
- ☐ Can explain: Why is plan-then-execute better than step-by-step for complex tasks?
- ☐ Can explain: What does the Critic check and why is it important?
- ☐ Can explain: What are guardrails and why are they necessary?

Next → Day 5: RAG + Vector Memory